

Fireflies.ai Clone

Meeting Notes & Transcription Platform

Complete Project Explanation

How it works · Every file & why it exists · Workflow · Database schema · API · Interview Q&A

Author: Jayant (jay12676)

Stack: Next.js (TypeScript) + FastAPI (Python) + SQLite

Repo: github.com/jay12676/FireFlies-ai-Clone

1. Project Overview

This is a functional clone of Fireflies.ai, a meeting-assistant web app. After a meeting, users browse a library of past meetings, open a meeting to read its interactive transcript (speaker labels + timestamps), view AI-generated notes (summary, action items, key topics), search across all transcripts, highlight and comment on transcript lines, ask questions about the meeting, and export the notes. Everything persists in a database.

What is real vs. mocked

- Real: the full post-meeting workflow — library, interactive transcript synced to a media player, AI notes, search, CRUD, highlights/comments, export, dark mode.
- Mocked (allowed by the brief): live meeting-bot recording and real speech-to-text. Transcripts are seeded, pasted, or uploaded (.txt / .vtt / .json).
- The 'AI' is a deterministic rule-based engine (no API key, no cost, fully offline) with a clean hook to swap in a real LLM later.

Core features delivered

- Meetings dashboard: search, filter by participant/tag, sort, responsive cards.
- Meeting detail: media player + interactive transcript with TWO-WAY sync.
- AI notes: summary (Gist), keywords, key topics/outline, action items.
- Full CRUD: create (form/paste/upload), edit, delete, action-item management.
- Bonus: global search, highlights/comments/soundbites, Ask-this-meeting chat, export to Markdown/TXT/PDF, dark mode, tags.

2. Tech Stack & Why

Frontend — Next.js 14 (App Router) + TypeScript + Tailwind CSS

Next.js gives file-based routing, fast rendering, and a first-class TypeScript experience. TypeScript catches type errors at build time (the API response shapes are typed end-to-end). Tailwind lets us build the polished Fireflies look quickly with utility classes and a dark-mode variant.

Backend — FastAPI (Python)

FastAPI is modern, async-capable, and auto-generates interactive API docs (Swagger at /docs) from Python type hints. Pydantic (built in) validates every request and serializes every response, so bad input is rejected before it reaches our logic.

ORM — SQLAlchemy 2.x · Database — SQLite

SQLAlchemy maps Python classes to tables and expresses relationships cleanly. SQLite is a zero-config, file-based database — perfect for a self-contained demo that runs anywhere with no separate DB server. The schema is standard SQL, so it would port to Postgres by changing one connection string.

Testing — pytest · Packaging — Docker

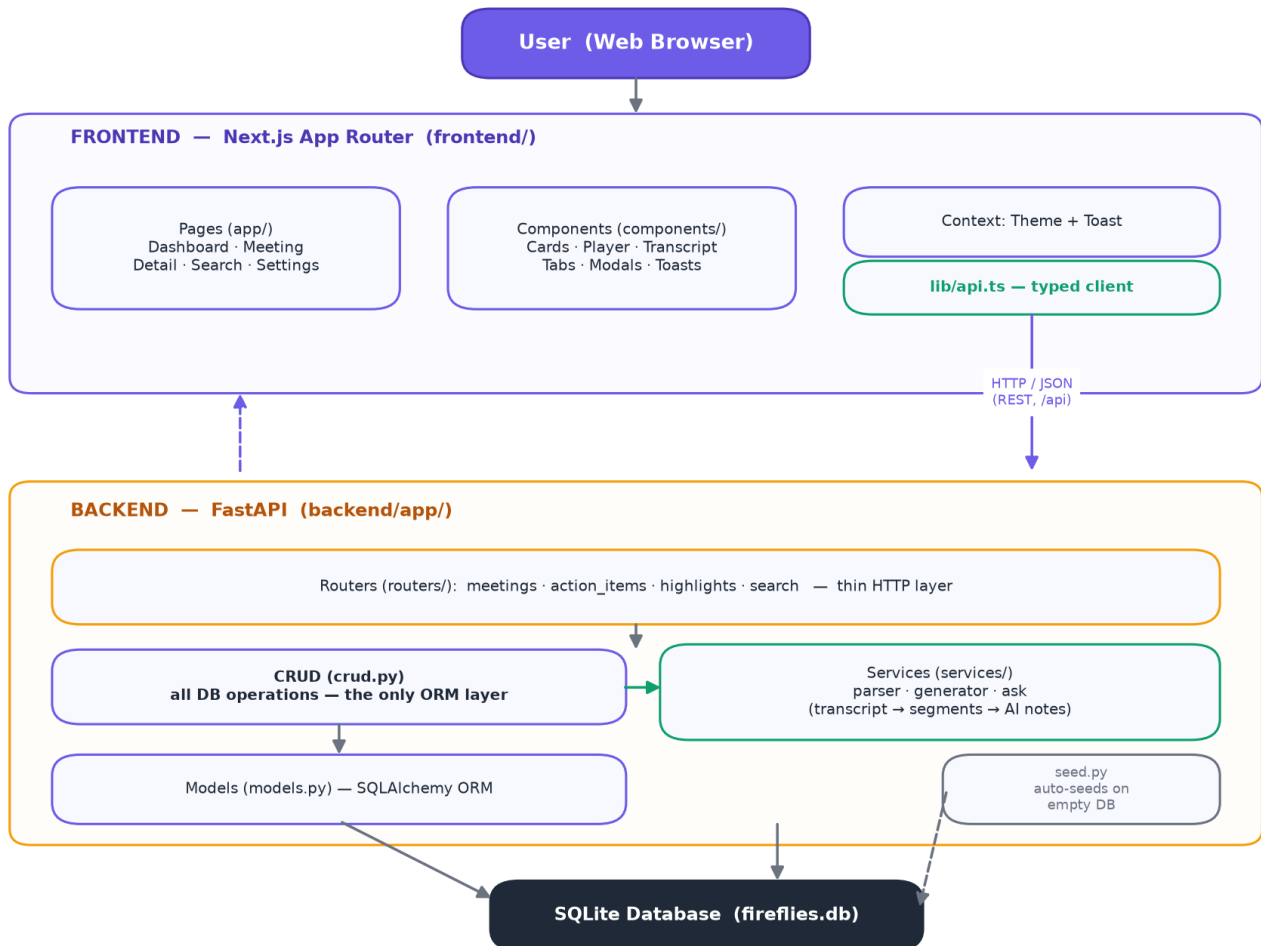
22 backend tests cover the parser, generator, models (cascade delete), and the full API lifecycle. Dockerfiles + docker-compose let the whole stack run with one command.

3. High-Level Architecture (HLD)

The system has three tiers with a strict one-directional dependency flow. The browser talks only to the frontend; the frontend talks to the backend only over HTTP/JSON; the backend is the only thing that touches the database.

Fireflies.ai Clone — High-Level Design (HLD)

Next.js (TypeScript) frontend · FastAPI (Python) backend · SQLite



Backend layering (separation of concerns)

- Routers (thin HTTP layer): validate input, call CRUD, return responses. No business logic. One router per resource.
- CRUD (crud.py): the ONLY layer that touches the ORM. All database reads/writes live here, so data access is centralised and testable.
- Services (parser, generator, ask): pure functions for transcript parsing and the rule-based AI. They know nothing about HTTP or the database.
- Models (SQLAlchemy) + Schemas (Pydantic): models are the DB tables; schemas are the request/response contracts. Keeping them separate means the API shape can differ from the storage shape.

Why this matters: each layer can be understood and tested on its own, and a change in one layer (e.g. swapping SQLite for Postgres, or the rule-based generator for an LLM) touches only that layer.

4. How a Request Flows (Walkthroughs)

A) Opening the dashboard

Browser loads `app/page.tsx` -> `lib/api.ts` calls `GET /api/meetings` -> the meetings router calls `crud.list_meetings()` -> SQLAlchemy runs one query (plus batched selectinload queries for participants/tags/action-items to avoid N+1) -> Pydantic serialises rows into `MeetingSummaryRow` JSON -> React renders the card grid.

B) Opening a meeting (the two-way sync)

`app/meetings/[id]/page.tsx` calls `GET /api/meetings/{id}` for the full detail. The page holds the `<audio>` element as the single source of truth for time:

- Click a transcript line / topic / citation -> `seekTo(ms)` sets `audio.currentTime` and updates `currentMs`.
- Audio playback -> `onTimeUpdate` fires -> `currentMs` updates -> `TranscriptPanel` highlights and auto-scrolls the line whose `[start_ms, next.start_ms)` window contains `currentMs`.
- Time is stored in milliseconds everywhere; we divide by 1000 only at the audio boundary (the HTML audio API uses seconds).

C) Creating / uploading a transcript

`POST /api/meetings` (or `/transcript`) sends the raw text + filename. The backend runs `services/parser.py` to turn it into ordered segments, then `services/generator.py` to produce the summary, action items, and topics, and stores everything. Re-uploading replaces the old segments and regenerates the AI notes so they never drift out of sync.

D) Asking a question about the meeting

`POST /api/meetings/{id}/ask` sends the question. `services/ask.py` classifies intent (action items / decision / who / general). Action-item questions return the extracted tasks; general questions score transcript segments by keyword overlap (with light stemming) and return the best matches with citations (speaker + timestamp) that the UI turns into clickable chips that seek the player.

5. Backend — Every File & Why It Exists

Folder: `backend/app/`

`main.py`

The FastAPI application factory. Creates the app, configures CORS (so the browser on :3000 may call the API on :8000), mounts all routers under `/api`, exposes `/api/health`, and auto-seeds sample data on startup if the DB is empty. Entry point run by `uvicorn`.

`database.py`

Sets up the SQLAlchemy engine, session factory (`SessionLocal`), declarative Base, and the `get_db()` dependency that hands each request a DB session and closes it afterwards. One place to configure the database URL.

`models.py`

The SQLAlchemy ORM models = the database tables (`Meeting`, `Participant`, `TranscriptSegment`, `Summary`, `KeyTopic`, `ActionItem`, `Highlight`, `Tag`). Defines columns, foreign keys, relationships, and cascade-delete rules.

`schemas.py`

Pydantic models = the API request/response contracts (Create/Update/Read variants). Validate incoming JSON and serialize outgoing JSON. Separate from models so the API shape is decoupled from storage.

`crud.py`

The data-access layer — the ONLY module that touches the ORM. Functions like `list_meetings`, `create_meeting`, `replace_transcript`, `add_action_item`, `global_search`, `ask_meeting`. Centralises all queries.

`routers/meetings.py`

HTTP endpoints for meetings: list, create, get, update, delete, upload transcript, add action item, add highlight, ask. Thin — delegates to `crud`.

`routers/action_items.py`

Standalone update/toggle-complete/delete endpoints for action items (PATCH/DELETE `/action-items/{id}`).

`routers/highlights.py`

Update/delete endpoints for highlights/comments (soundbites).

`routers/search.py`

GET `/search?q=` — global full-text search across all transcript segments.

`services/parser.py`

Converts an uploaded `.txt` / `.vtt` / `.json` transcript into a uniform list of segments (speaker, start_ms, end_ms, text). Picks the handler by extension, falls back to content sniffing, synthesises timestamps when missing.

`services/generator.py`

The rule-based AI: extractive summary (rank sentences by keyword frequency), action-item extraction (cue phrases like 'will', 'need to', 'by Friday'), and topic clustering (frequent keywords anchored to timestamps). Deterministic — same input always gives the same output.

`services/ask.py`

The rule-based 'ask this meeting' engine. Classifies the question's intent, then either returns structured data (action items) or retrieves the most relevant transcript segments by keyword overlap with citations.

`seed.py`

Builds 5 realistic sample meetings (standup, roadmap, sales call, 1:1, retro) with full transcripts, summaries, topics, action items, participants, tags. Auto-runs on first startup so the app is usable immediately.

`tests/`

pytest suite: `test_parser`, `test_generator`, `test_models` (cascade), `test_api` (full CRUD + search + highlights + ask). 22 tests.

6. Frontend — Every File & Why It Exists

Folder: `frontend/`

`app/layout.tsx`

The root shell wrapping every page: sidebar + navbar + Theme and Toast context providers. Rendered once around all routes.

`app/page.tsx`

The dashboard / meetings library: card grid with debounced search, participant & tag filters, sort, loading skeletons, and the New-Meeting modal.

`app/meetings/[id]/page.tsx`

The meeting detail page. Owns the audio element and the `currentMs/seekTo` state that drives the two-way transcript sync; wires the player, transcript, notes tabs, edit/delete, export, and highlights.

`app/search/page.tsx`

Global search results grouped by meeting with highlighted snippets.

`app/settings/page.tsx`

Profile, dark-mode toggle, and 'Coming Soon' placeholders (bot, integrations, team, live transcription).

components/

Reusable UI units: Sidebar, Navbar, MeetingCard, Avatars, Modal, NewMeetingModal, EditMeetingModal, MediaPlayer, TranscriptPanel, MeetingTabs, ActionItems, AskChat, ExportMenu, Highlight. Each has one responsibility.

context/ThemeContext.tsx

Dark-mode state, persisted to localStorage and applied as a class on <html>.

context/ToastContext.tsx

App-wide toast notifications via a simple queue + portal.

lib/api.ts

The single typed API client. Every backend call goes through one request() helper (base URL, JSON headers, error unwrapping). Also holds date/time/duration formatting helpers.

lib/types.ts

TypeScript interfaces mirroring the backend schemas — end-to-end type safety.

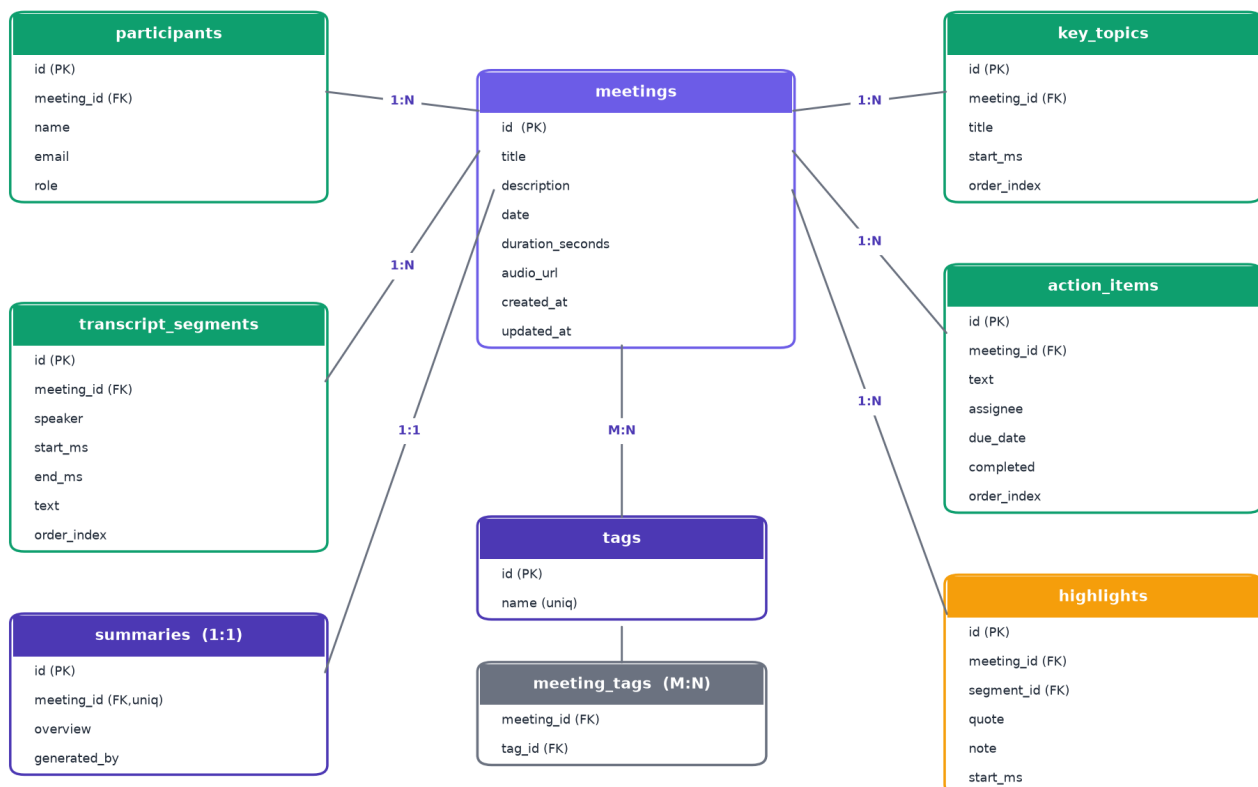
lib/export.ts

Builds Markdown / TXT / PDF exports of a meeting (jsPDF for the PDF), all client-side.

7. Database Schema

Meetings is the central table. Everything else hangs off it. All child rows cascade-delete when their meeting is deleted, so there are never orphan records.

Fireflies.ai Clone — Database Schema (ER Diagram)



All child rows CASCADE-DELETE when their meeting is deleted.

Relationships & why

- Meeting 1:N Participant / TranscriptSegment / KeyTopic / ActionItem / Highlight — a meeting has many of each.
- Meeting 1:1 Summary — each meeting has exactly one summary (enforced by a UNIQUE constraint on summaries.meeting_id).
- Meeting M:N Tag via the meeting_tags join table — a tag can belong to many meetings and a meeting can have many tags.
- Highlight -> TranscriptSegment (nullable FK) — a highlight snapshots the quote/speaker/time so the saved soundbite survives even if the transcript is later replaced.
- start_ms / end_ms on segments (milliseconds) drive the player seek and the active-line highlight; order_index preserves display order.

8. API Reference (prefix /api)

REST conventions: nouns for resources, HTTP verbs for actions, correct status codes (201 created, 204 no-content, 404 not found). Interactive docs live at <http://localhost:8000/docs>.

GET	/meetings	List meetings (?search, participant, tag, sort)
POST	/meetings	Create meeting (+ optional pasted/uploaded transcript)
GET	/meetings/{id}	Full meeting detail
PATCH	/meetings/{id}	Update title / participants / tags
DELETE	/meetings/{id}	Delete meeting (cascades to children)
POST	/meetings/{id}/transcript	Upload/paste transcript -> parse -> regenerate
POST	/meetings/{id}/action-items	Add an action item
PATCH	/action-items/{id}	Edit / toggle complete
DELETE	/action-items/{id}	Delete an action item
POST	/meetings/{id}/highlights	Add a highlight / comment (soundbite)
PATCH	/highlights/{id}	Edit a highlight
DELETE	/highlights/{id}	Delete a highlight
POST	/meetings/{id}/ask	Ask a question -> answer + cited segments
GET	/search?q=	Global search across all transcripts

9. The 'AI' Explained (Rule-Based, No LLM)

A common interview question: 'how does the AI work?' The answer is honest — it is classic NLP heuristics, chosen deliberately so the demo is deterministic, free, and works offline. Each piece has a clean seam to swap in a real LLM.

Summary — extractive, frequency-based

Count how often each meaningful word appears (after removing stopwords). Score each sentence by the average frequency of its words, take the top N, and re-sort them into reading order. This is the idea behind Luhn's algorithm / TextRank.

Action items — cue-phrasе matching

Scan sentences for action cues ('will', 'I'll', 'need to', 'let's', 'follow up', 'action item', 'by Friday', ...). The nearest preceding speaker becomes the assignee; a date phrase becomes the due date.

Topics / outline — keyword clustering

Take the most frequent meaningful keywords and anchor each to the first timestamp where it appears, producing time-linked chapters.

Ask this meeting — intent routing + retrieval

Classify the question (action / decision / who / general). Action questions return the structured action items; general questions score segments by keyword overlap (with light stemming so 'payments' matches 'payment') and return the best matches with timestamped citations.

Upgrade path: every one of these is a single pure function. To use a real LLM, replace the body of that function with an API call that sends the same retrieved context — nothing else in the app changes.

10. Key Design Decisions & Trade-offs

- Layered backend (routers -> crud -> models + services): separation of concerns; each layer is independently testable.
- Rule-based AI instead of an LLM: deterministic, free, offline, and reproducible for a graded demo — with a clear upgrade seam.
- SQLite: zero-config and portable; standard SQL means an easy move to Postgres.
- Milliseconds everywhere for time: integer math avoids floating-point drift; we convert to seconds only at the HTML audio boundary.
- Naive local datetimes for display: the shown clock time never shifts due to timezone conversion. New meetings capture the live creation time.
- Single typed API client on the frontend: base URL, headers, and error handling live in one place; components never call fetch() directly.
- Replace-and-regenerate on transcript upload: keeps AI notes consistent with the actual words; makes the parse->generate pipeline idempotent.

11. Likely Interview Questions & Answers

Q. Walk me through what happens when I click a transcript line.

A. The line's onClick calls seekTo(startMs), which sets the shared <audio> element's currentTime and plays it. As it plays, onTimeUpdate updates currentMs; TranscriptPanel recomputes which segment is active (last one with start_ms <= currentMs) and highlights + auto-scrolls it. That's the two-way sync.

Q. How is your database structured and why?

A. Meetings is the central table; participants, transcript_segments, key_topics, action_items, and highlights are 1:N children; summaries is 1:1; tags is M:N via a join table. Children cascade-delete with their meeting so there are no orphans.

Q. Why FastAPI and how is the backend organised?

A. FastAPI gives type-driven validation and auto docs. It's layered: routers handle HTTP, crud.py is the only ORM layer, services hold parsing + the rule-based AI, models are the tables, schemas are the API contracts.

Q. How does the AI summary work? Is it a real model?

A. It's deterministic rule-based NLP: extractive frequency-based summary, cue-phrase action items, keyword-clustered topics. I chose it so the demo is free, offline, and reproducible, and I left each function as a clean seam to drop in an LLM.

Q. How do you avoid the N+1 query problem?

A. In `crud.list_meetings` I use SQLAlchemy `selectinload` for `participants/tags/action_items`, so related rows are fetched in a few batched queries instead of one query per meeting.

Q. What happens on transcript upload?

A. The parser normalises `.txt/.vtt/.json` into uniform segments; then the generator rebuilds the summary, topics, and action items. I replace the old data entirely so the notes can't drift from the transcript.

Q. How would you scale or productionise this?

A. Swap SQLite for Postgres (one connection string), add authentication (JWT), move the rule-based services to real LLM calls behind the same function signatures, add caching, and deploy the frontend (Vercel) and backend (Render/Railway) — Dockerfiles are already provided.

Q. How is the frontend kept type-safe and DRY?

A. `lib/types.ts` mirrors the backend schemas, and every network call goes through one typed `request()` helper in `lib/api.ts`, so base URL, headers, and error handling exist in exactly one place.

12. How to Run & Test

Backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate          # Windows
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
# API: http://localhost:8000    Docs: http://localhost:8000/docs
```

Frontend

```
cd frontend
cp .env.example .env.local      # NEXT_PUBLIC_API_URL=http://localhost:8000
npm install
npm run dev                     # http://localhost:3000
```

Tests / seed

```
cd backend
pytest                          # 22 tests
python -m app.seed --reset      # wipe & reseed sample meetings
```